



University of Bologna
Dipartimento di Informatica –
Scienza e Ingegneria (DISI)
Engineering Bologna Campus

Class of ICCBD
Infrastructures for
Cloud Computing and Big Data M

Data Batching

Antonio Corradi
Academic year 2023/2024

BIG DATA

**We have to deal with an enormous amount of data
And we have to change our mind setting
(moving from the small perspective toward a more
significant and effective way of processing data)**

**It is most important to change our normal analysis
to suit the new requirements and strategies**

Which are the requirements???

Which are the operations on Big Data?

ANALYSIS OF BIG DATA

SETTINGS for the specific pattern **in the large**
VERY COMMON → easy to be offered

- **Big Data storage, access and management**
- **Data batch processing**
- **Stream data processing**
-

Many other patterns of work, less common

Data and any service available on them **as the input**

BIG DATA SETS AND ENVIRONMENTS

MAIN GENERAL PROPERTIES <> enormous data volume

- Distribution & Decentralization
- Scalability
- Efficiency
- Quality of Service

MAIN SYSTEM REQUIREMENTS for SUPPORT

- Long life cycle ($\rightarrow \infty$)
- Open source
- Interoperability and standard (no lock in)
- Remote control (dashboard for monitoring)
- Transparency (black box) and Visibility

BATCH DATA PROCESSING IN LARGE CLUSTERS

Often it is paramount to **process automatically** a very large set of data of a specified dimension so to provide **fast results for a search**

Very common in **big data installations**

Map-Reduce batch published in 2004

Excellent mechanism for obtaining a **high throughput** result in a **scalable, reliable, and maintainable** way

Pioneer examples

UNIX batch: simple log analysis at system level

- It is common to use **input as immutable** and produce **output on need**
- In **UNIX batch**, parallelism was not a primary goal (data not so large)

DATA PROCESSING IN TODAY LARGE CLUSTERS

Excellent data parallelism

- It is easy to find what to parallelize
*Example: **web data crawled by Google** that need to be indexed – **documents** can be analyzed independently*
- It is common to use **thousands of nodes** for one program that processes **an enormous amounts of data (big data)**

Communication overhead not so significant compared with the overall execution time

- Tasks access **to disks frequently** and sometimes run complex algorithms – **access to data & computation time** dominates the execution time

Data access rate can become the **bottleneck**

HPC CONTEXT BSP

Automatize everything – for useful **special-purpose** behavior but widening the potential user audience

Bulk Synchronous Parallel as a way to model easily the internal execution and the coordination of these processes

Highly Parallel Computing (HPC) makes large use of **special purpose languages** (very esoteric, weird, and not so spread) to solve parallelism

- To propose some solution **patterns**, HPC proposed a simplified way of dealing and commanding **several processes in parallel at the same time**
- **users express only** the phases of **local computation**, **inter-communication**, and a barrier of **synchronization**

DATA PROCESSING IN TODAY LARGE CLUSTERS



Engineers can **focus only on the application logic and parallel tasks**, without dealing with scheduling, fault-tolerance, and synchronization

MapReduce is a **programming framework** that provides

- **High-level API** to specify **parallel tasks**
- **Runtime system** takes care of
 - Automatic parallelization & scheduling
 - Load balancing
 - Fault tolerance
 - I/O scheduling
 - Monitoring & status updates
- Everything runs on top of **GFS** (the distributed file system)

MAP-REDUCE DETAILS

Automatize everything – for useful **special-purpose** behavior in **two steps of complementary operations of Map-Reduce**

Two phases cleanly defined and specified in 2 successive steps:
MAP and **REDUCE**

- The **MAP** first phase has to work on the data by splitting **them in chunks to be operated upon in parallel**
- The **REDUCE** second phase has to work on the collection of data and in putting them together **toward the final unified results**
- **Intermediate data are stored on the GFS**
- **Processes are dynamically decided**

USER BENEFITS OF BATCHING MAP-REDUCE

Automatize everything – for useful **special-purpose** behavior in **two steps of complementary operations of Map-Reduce**

- Based on **abstract black box** approach
- Huge **speedups** in programming/prototyping
«it makes it possible to write a simple program and run it efficiently on a thousand machines in half an hour»
- Designers can exploit **quite easily very large amounts of resources**
 - Including **users with no experience** in distributed / parallel systems

TRADITIONAL MAPREDUCE DEFINITIONS

Statements that go back to **functional languages** (such as LISP, Scheme) as a **sequence of two steps** for **parallel exploration and results** (**Map** and **Reduce**)

Also in other programming languages: **Map/Reduce** in Python, in Perl, in Java, ...

Map (distribution phase and the parallelize decision)

1. **Input:** a *list of data* and one *function*
2. **Execution:** the function **is applied to** each list item
3. **Result:** a *new list* with all the results of the function

Reduce (result harvesting phase)

1. **Input:** a *list* and one *function*
2. **Execution:** the function **combines/aggregates** the list items
3. **Result:** one **new final item**

WHAT IS MAPREDUCE IN A NUTSHELL

The terms are borrowed from Functional Languages (e.g., Lisp)

Sum of squares:

`(map square `(1 2 3 4)) => Output: (1 4 9 16)`

[processes each record **sequentially and independently**]

`(reduce + '(1 4 9 16)) => (+ 16 (+ 9 (+ 4 1) ) ) => Output: 30`

[processes set of **all records in batches**]

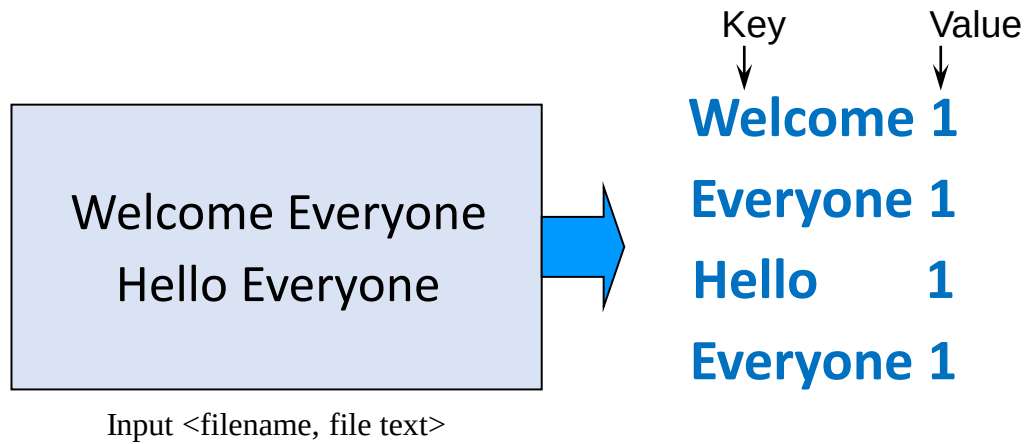
Let us consider a sample application: [Wordcount](#)

You are given a **huge dataset** (e.g., Wikipedia dump – or all of Shakespeare's works) and asked to list **the count for each of the words in any searched document**

MAP

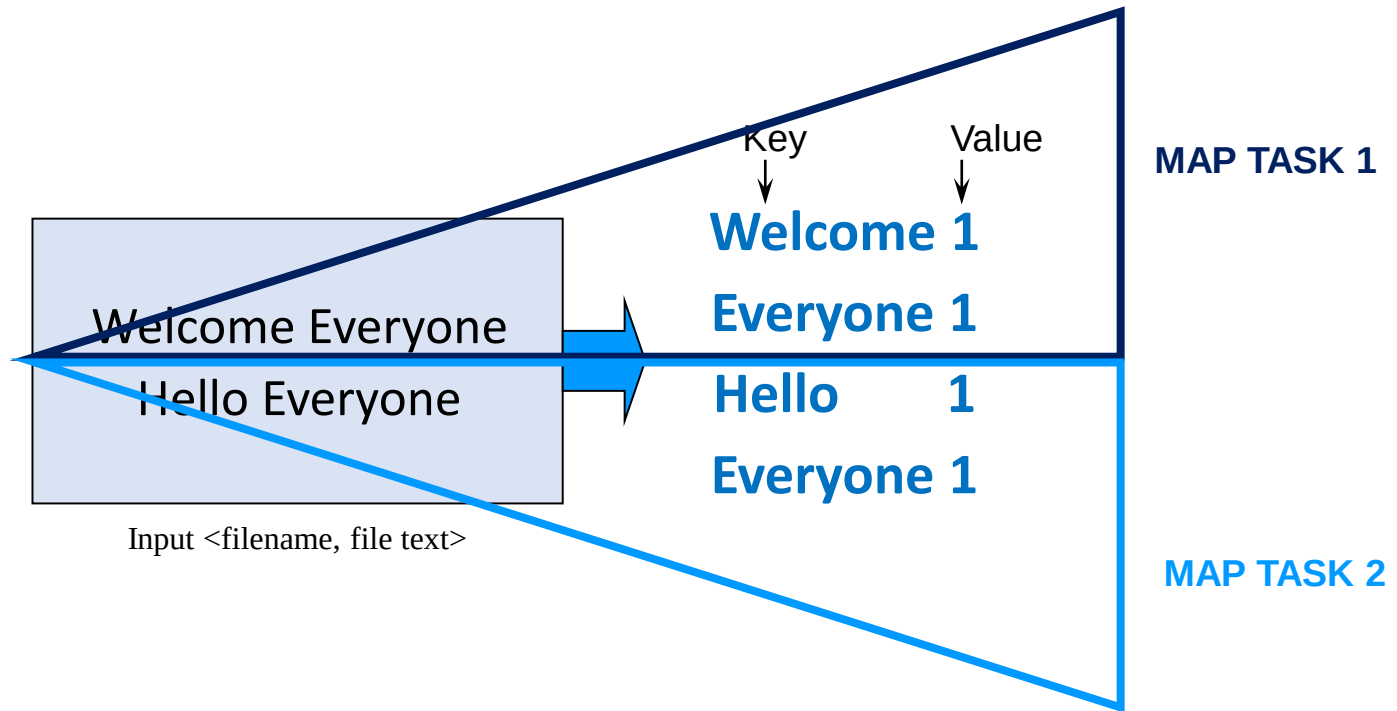
Extensively apply the function

Process all single records to generate intermediate key/value pairs.



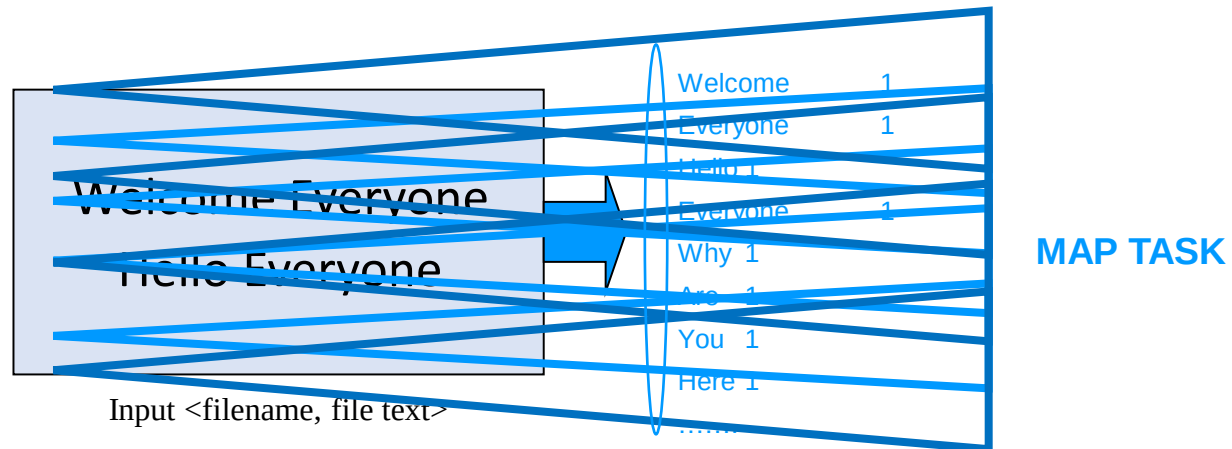
MAP

In parallel process individual records to generate intermediate key/value pairs



MAP

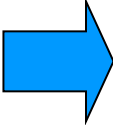
In parallel process **a large number** of individual records to generate intermediate key/value pairs



REDUCE

Collect the whole information

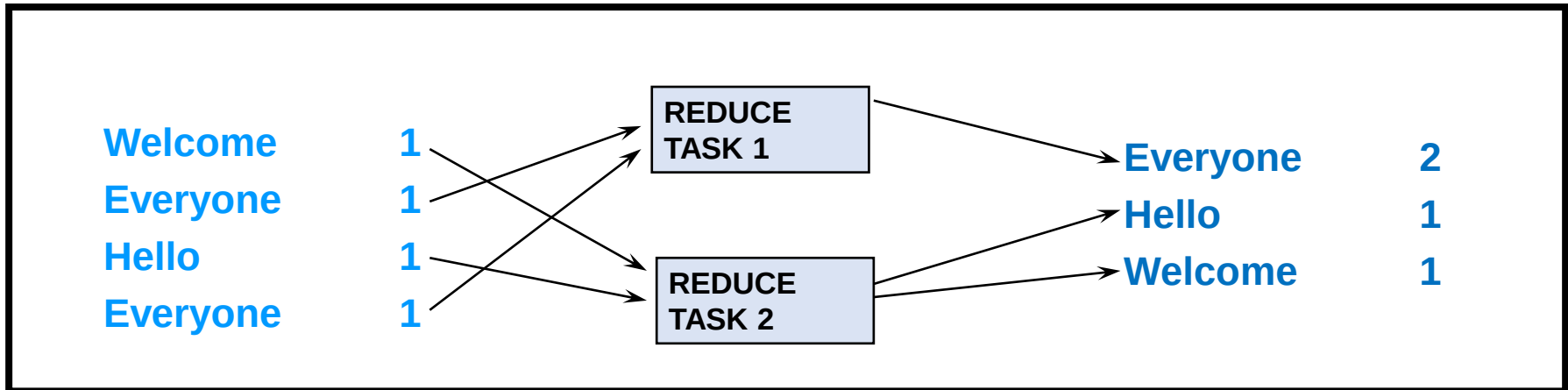
Reduce processes and **merges all intermediate values**
associated **per key**

				Key	Value
				↓	↓
Welcome	1			Everyone	2
Everyone	1			Hello	1
Hello	1			Welcome	1
Everyone	1				

REDUCE

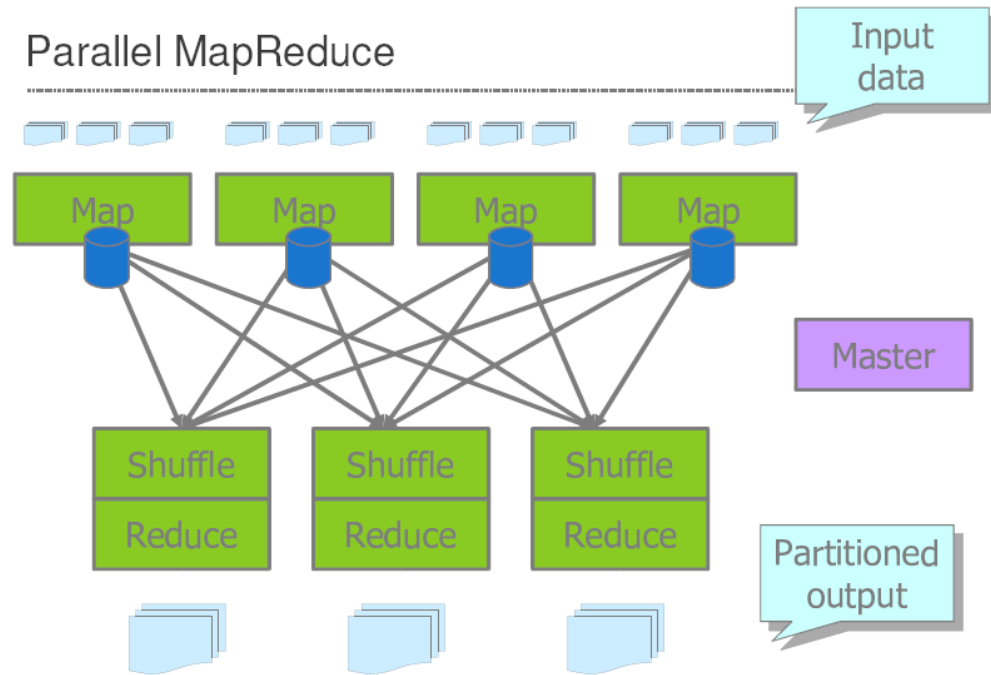
Each key assigned to one Reduce

- **In parallel** processes and merges all intermediate values by partitioning keys



- **Popular splitting: *Hash partitioning***, such as key is assigned to
 - $\text{reduce \#} = \text{hash}(\text{key}) \% \text{number of reduce tasks}$

MAPREDUCE: A DEPLOYMENT VIEW



Read many chunks of distributed data (no data dependencies)

Map: extract something from each chunk of data

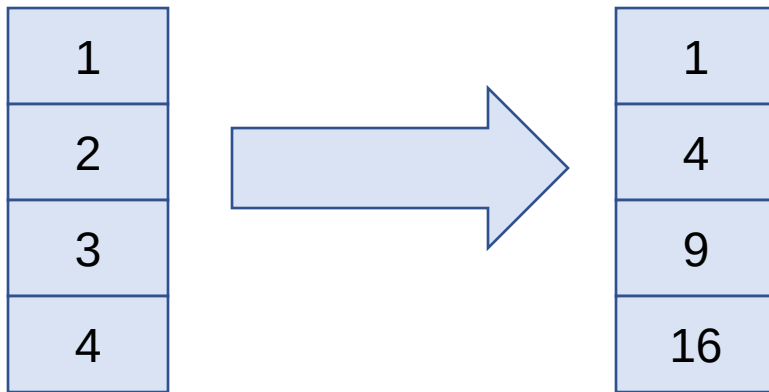
Shuffle and sort

Reduce: aggregate, summarize, filter or transform sorted data

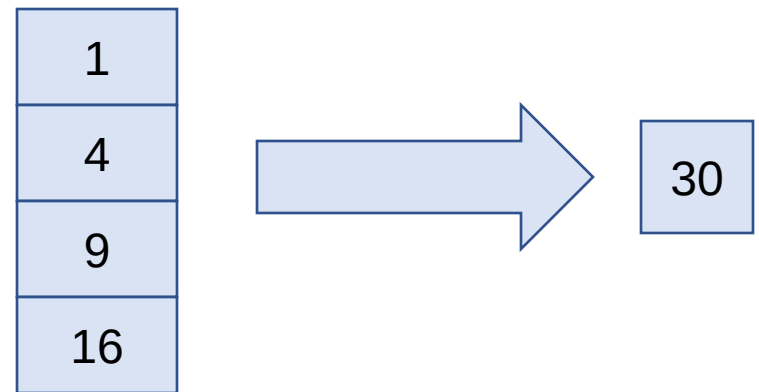
Programmers can specify the **Map** and **Reduce functions**

TRADITIONAL MAPREDUNDANCE EXAMPLES (AGAIN)

Map (square, [1, 2, 3, 4])



Reduce (add, [1, 4, 9, 16])



GOOGLE MAPREDUCE DEFINITION

map (String key, String val) runs on each item in the set

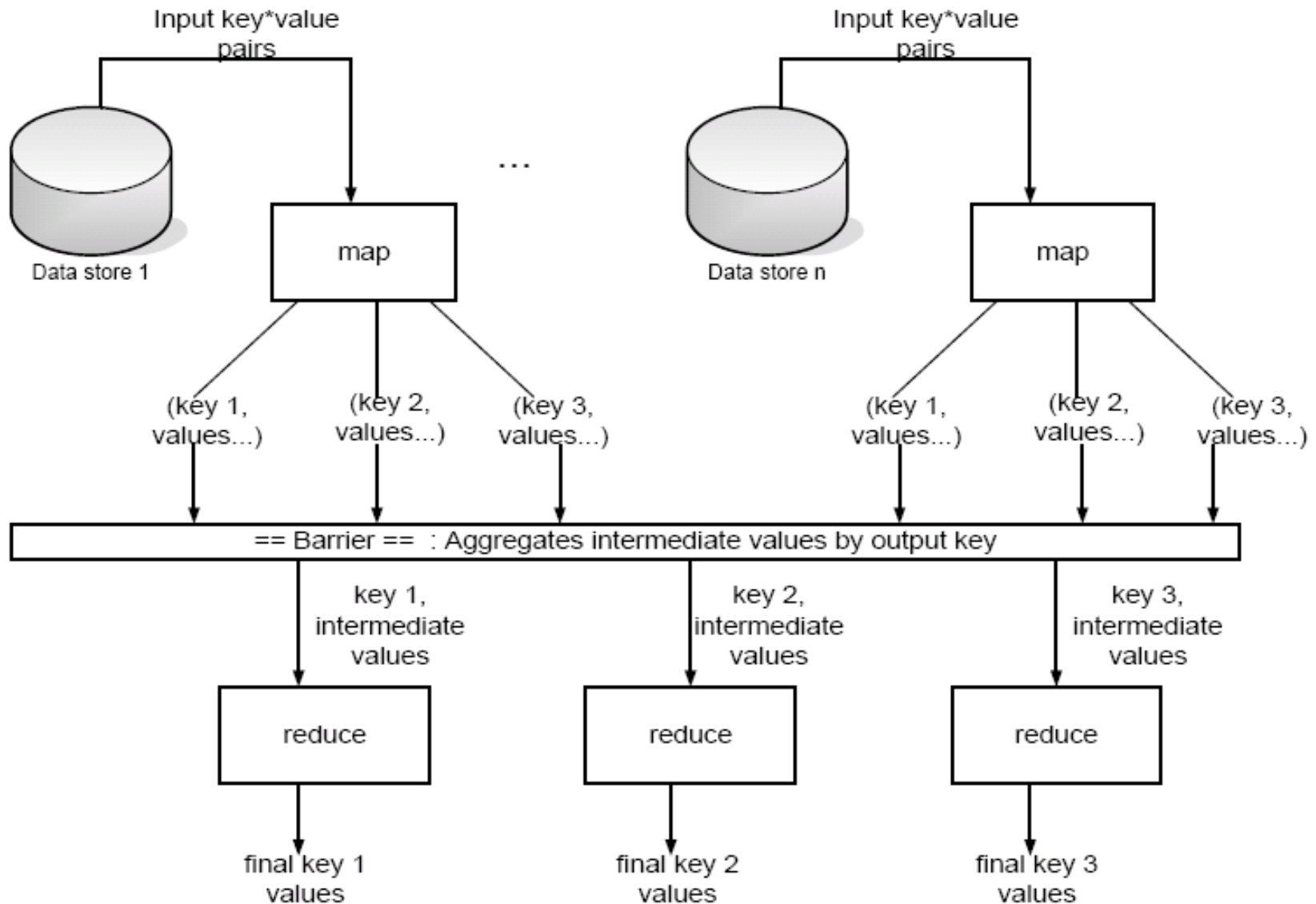
- **Input example:** a set of files, with keys being **file names** and values being **file contents**
- **Keys & values** can have different types: the programmer has to convert between Strings and appropriate types inside map()
- **Emits**, i.e., outputs, (new-key, new-val) pairs
- Size of output set can be different from size of input set

The runtime system **aggregates the output of map by key**

reduce (String key, Iterator vals) runs for each *unique* key emitted by map()

- It is possible to have more values for one key
- **Emits final output pairs** (possibly smaller set than the intermediate sorted set)

MAP & AGGREGATION MUST FINISH BEFORE REDUCE CAN START



RUNNING A MAPREDUCE PROGRAM

The final user fills in *specification object*:

- *Input/output file names*
- *Optional tuning parameters*
(e.g., size to split input/output into)

The final user defines *MapReduce function* and passes it *the specification object*

The **runtime system** calls `map ()` and `reduce ()`

- While the final user just has to specify the operations

WORD COUNT EXAMPLE

```
map(String input_key, String input_value):
```

```
// input_key: document name
```

```
// input_value: document contents
```

```
  for each word w in input_value:
```

```
    EmitIntermediate(w, "1");
```

```
reduce(String output_key,  
        Iterator intermediate_values):
```

```
// output_key: a word
```

```
// output_values: a list of counts
```

```
  int result = 0;
```

```
  for each v in intermediate_values:
```

```
    result += ParseInt(v);
```

```
  Emit(AsString(result));
```

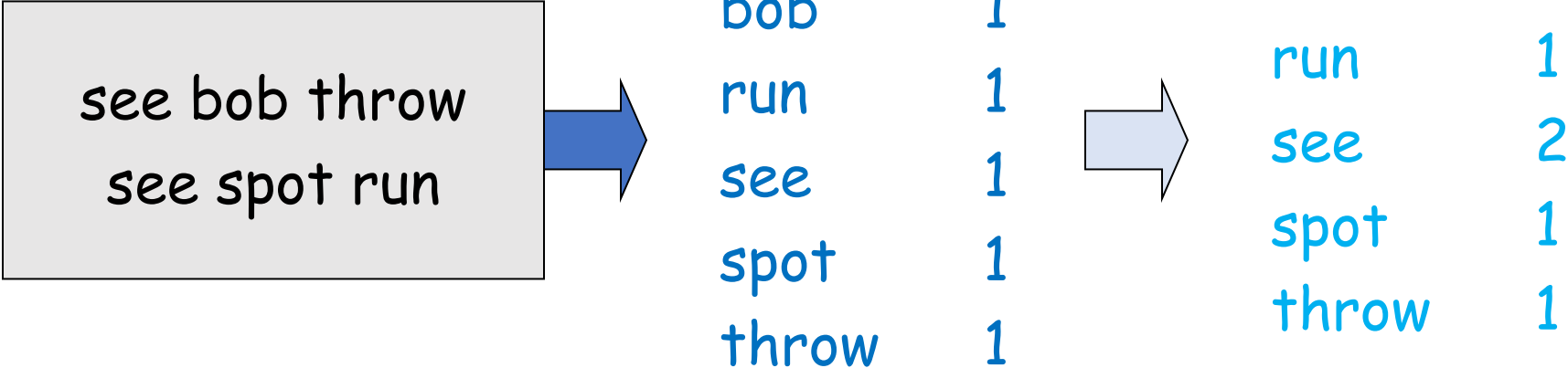
WORD COUNT ILLUSTRATED

`map(key=url, val=contents):`

- For each word `w` in `contents`, emit `(w, "1")`

`reduce(key=word, values=uniq_counts):`

- Sum all `"1"`s in values list
- Emit result `"(word, sum)"`



see bob throw
see spot run

see 1
bob 1
run 1
see 1
spot 1
throw 1

bob 1
run 1
see 2
spot 1
throw 1

MANY OTHER APPLICATIONS

Distributed *grep*

- map() emits a line if it matches a supplied pattern
- reduce() is an identity function; just emit same line

Distributed *sort*

- map() extracts *sorting key* from record (file) and outputs (key, record) pairs
- reduce() is an identity function; just emit same pairs
- The actual sort is done automatically by runtime system

Reverse web-link graph

- map() emits (*target*, *source*) pairs for each link to a *target* URL found in a file *source*
- reduce() emits pairs (*target*, list(*source*))

OTHER APPLICATIONS

- **Machine learning** issues
- Google **news clustering** problems
- **Extracting data** + reporting **popular queries** (Zeitgeist)
- Extract **properties** of web pages for tests/products
- Processing satellite imagery data
- Graph computations
- Language model for machine translation

Rewrite of Google Indexing Code in MapReduce

Used until 2011, then adopted other repositories technologies

IMPLEMENTATION OVERVIEW (AT GOOGLE)

Environment:

Large clusters of PCs connected with Gigabit links

- 4-8 GB RAM per machine, dual x86 processors
- Network bandwidth often significantly less than 1 GB/s
- Machine failures are common due to # machines

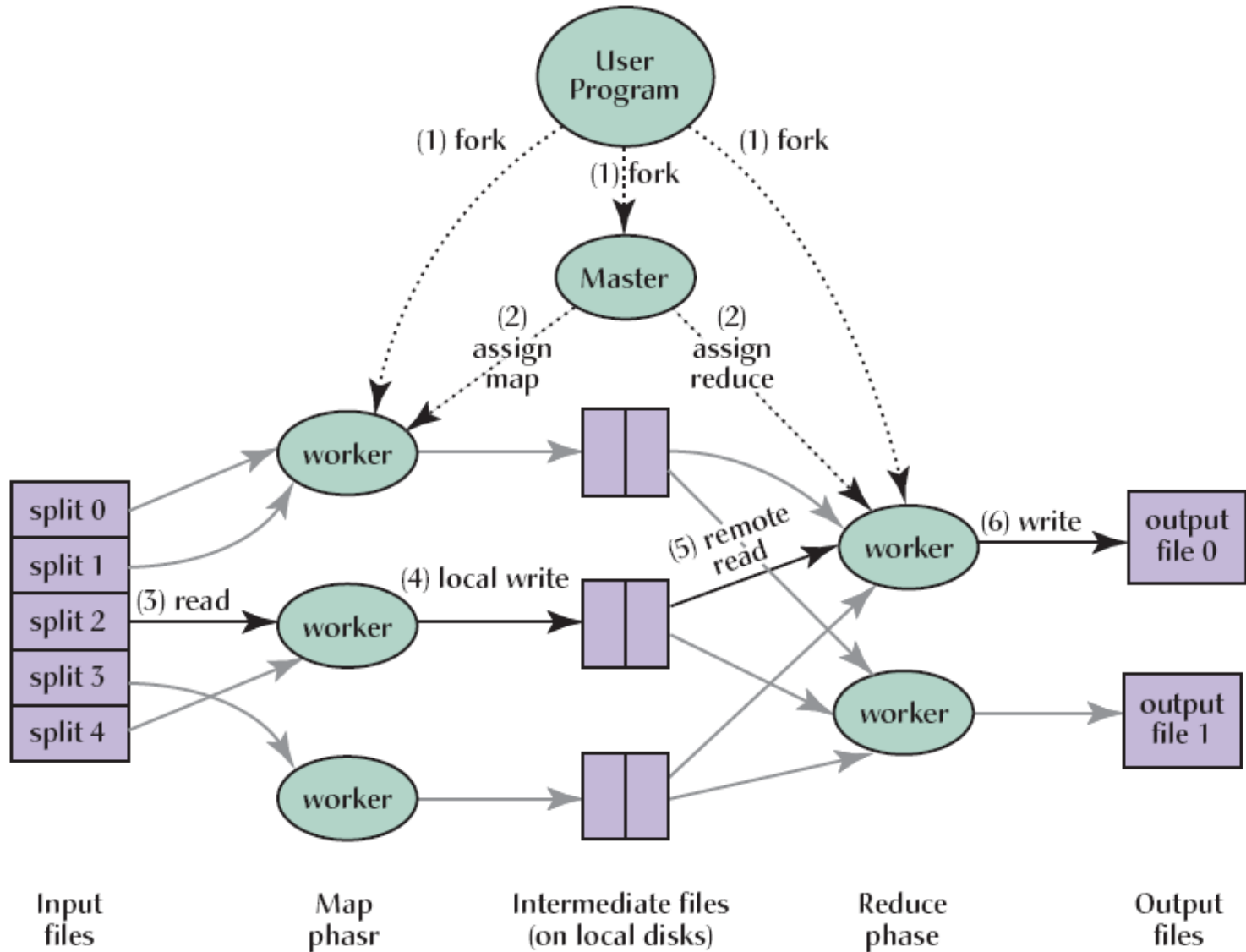
GFS: distributed file system manages data

- Storage is provided by cheap IDE disks attached to machine

Job scheduling system: jobs made up of tasks, scheduler assigns tasks to machines

Implementation is a C++ library linked into user programs

ARCHITECTURE EXAMPLE



SCHEDULING AND EXECUTION

One master, many workers

- Input data split into M map tasks (typically 64 MB in size)
- Reduce phase partitioned into R reduce tasks
- Tasks are assigned to workers dynamically
- Often: M=200,000; R=4000; workers=2000

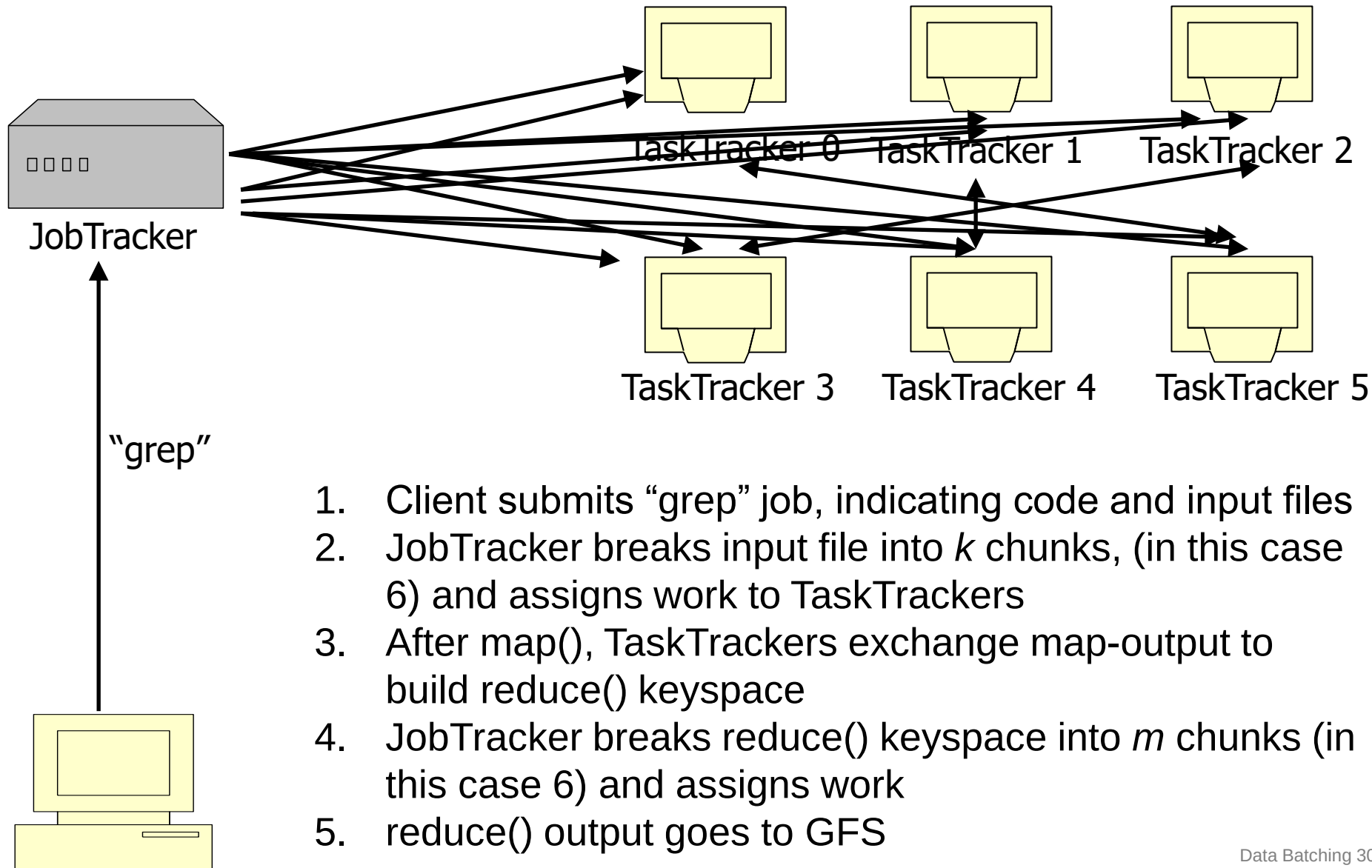
Master assigns each map task to a free worker

- Consider locality of data to worker when assigning a task
- Workers read task input (often from local disk)
- Intermediate key/value pairs written to **local disk**, divided into **R regions**, and the locations of the regions are passed to the master

Master assigns each reduce task to a free worker

- Worker reads intermediate k/v pairs from map workers
- Worker applies user reduce operation to produce the output (stored in GFS)

SCHEDULING AND EXECUTION EXAMPLE (2)



FAULT-TOLERANCE

On master failure:

- State is checkpointed to GFS: new master recovers & continues

On worker failure:

- Master detects failure via **periodic heartbeats**
- Both **completed and in-progress map tasks** on that worker should be re-executed (→ output stored on local disk)
- Only **in-progress reduce tasks** on that worker should be re-executed (→ output stored in global file system)

Robustness:

- Example: Lost 1600 of 1800 machines once, but still success

FAVOURING DATA LOCALITY

The goal is **to preserve and to conserve network bandwidth**

- **GFS** has data files divided into 64 MB blocks and 3 copies of each are stored on different machines
- Master program **schedules map() tasks based on the location of these replicas:**
 - Put **map() tasks** physically on the **same machine** as one of the input replicas (or, at least on the same rack/network switch)
 - In this way, the machines can read input at local disk speed
Otherwise, rack switches would limit read rate

BACKUP TASKS



Problem: stragglers (i.e., **slow workers in finishing**) significantly lengthen the completion time

- Other jobs may be consuming resources on machine
- Bad disks with soft errors (i.e., correctable) transfer data very slowly
- Other weird things: processor caches disabled at machine init

Solution: Close to completion, **spawn backup copies of the remaining in-progress tasks**

- Whichever one finishes first, wins
- Additional cost: a few percent more resource usage
- Result: A sort program without backup was 44% longer

HADOOP: A JAVA-BASED MAPREDUCE



Hadoop is an **open source platform for MapReduce by Apache**

Started as **open source MapReduce written in Java**, but evolved to support other Apache languages such as Pig and Hive

Hadoop common

set of utilities that support the other subprojects:

- FileSystem, RPC, and serialization libraries

Several essential subprojects:

- **Distributed file system (*HDFS*)**
- ***MapReduce***
- **Yet Another Resource Negotiator (YARN)** for cluster resource management

YARN RESOURCE MANAGER

YARN provides **management for virtual Hadoop clusters** over a large physical cluster

- Handles **node allocation** in a cluster
- Supplies **new nodes** with configuration
- **Distributes** Hadoop to allocated nodes
- **Starts** Map/Reduce and HDFS workers
- **Includes** management and monitoring

Today, other resource managers are available, such as MESOS



Apache
MESOS™

VS.



THE YARN SCHEDULER

YARN or **Yet Another Resource Negotiator**

Used underneath **Hadoop** 2.x +

Treats **each server** as a collection of *containers*

- Container = fixed CPU + fixed memory
(think of Linux cgroups, but even more lightweight)

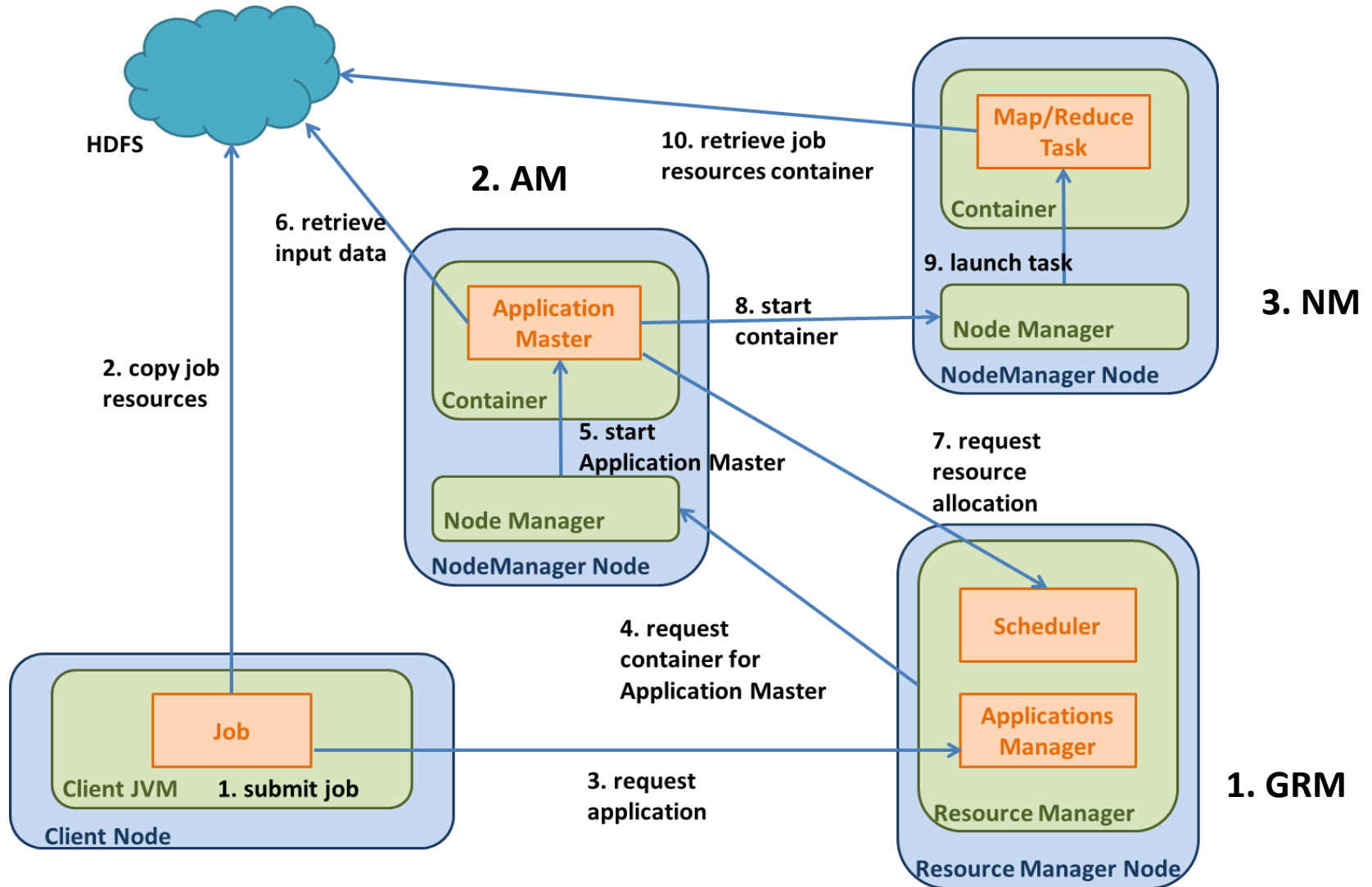
THE YARN SCHEDULER

YARN or **Yet Another Resource Negotiator**

Main components

1. **Global Resource Manager (GRM)** node
 - **Scheduler** that globally allocates the required resources
 - **ApplicationManager** that coordinates the execution of the job on the other nodes
2. **Application Master (AM) Per-application** (per job)
 - Container negotiation with RM and NMs
 - Detecting task failures of that job
3. **Per-server Node Manager (NM)**
 - Daemon and server-specific functions that **manage** local resources, **instantiate containers** to run tasks, **monitor container** resource usage

YARN AT WORK



HADOOP EXTENSIONS (OUT-OF-OUR-SCOPE)

- **Avro**: Large-scale data serialization
- **Chukwa**: Data collection (e.g., logs)
- **Hbase**: Structured data storage for large tables
- **Hive**: Data warehousing and management (Facebook)
- **Pig**: Parallel SQL-like language (Yahoo)
- **ZooKeeper**: coordination for distributed apps
- **Mahout**: machine learning and data mining library
- **Sahara**: deployment of Hadoop clusters on OpenStack

HADOOP FOR OPENSTACK

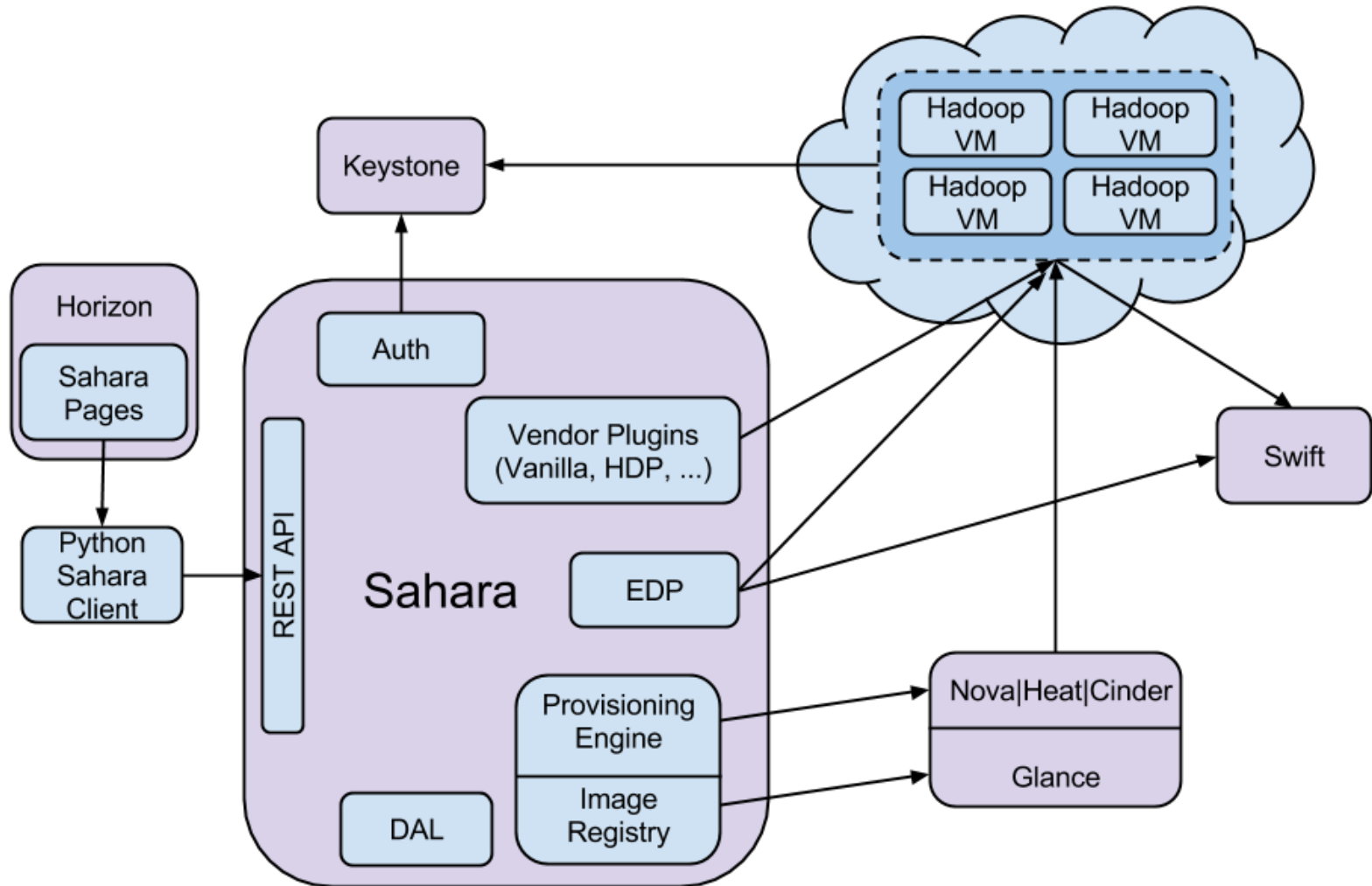
Hadoop can exploit the virtualization provided by **OpenStack** in order to obtain more flexible clusters and a better resource utilization

OpenStack **Sahara service** can allow **deploying** and **configuring Hadoop clusters** in a **Cloud environment by adding services:**

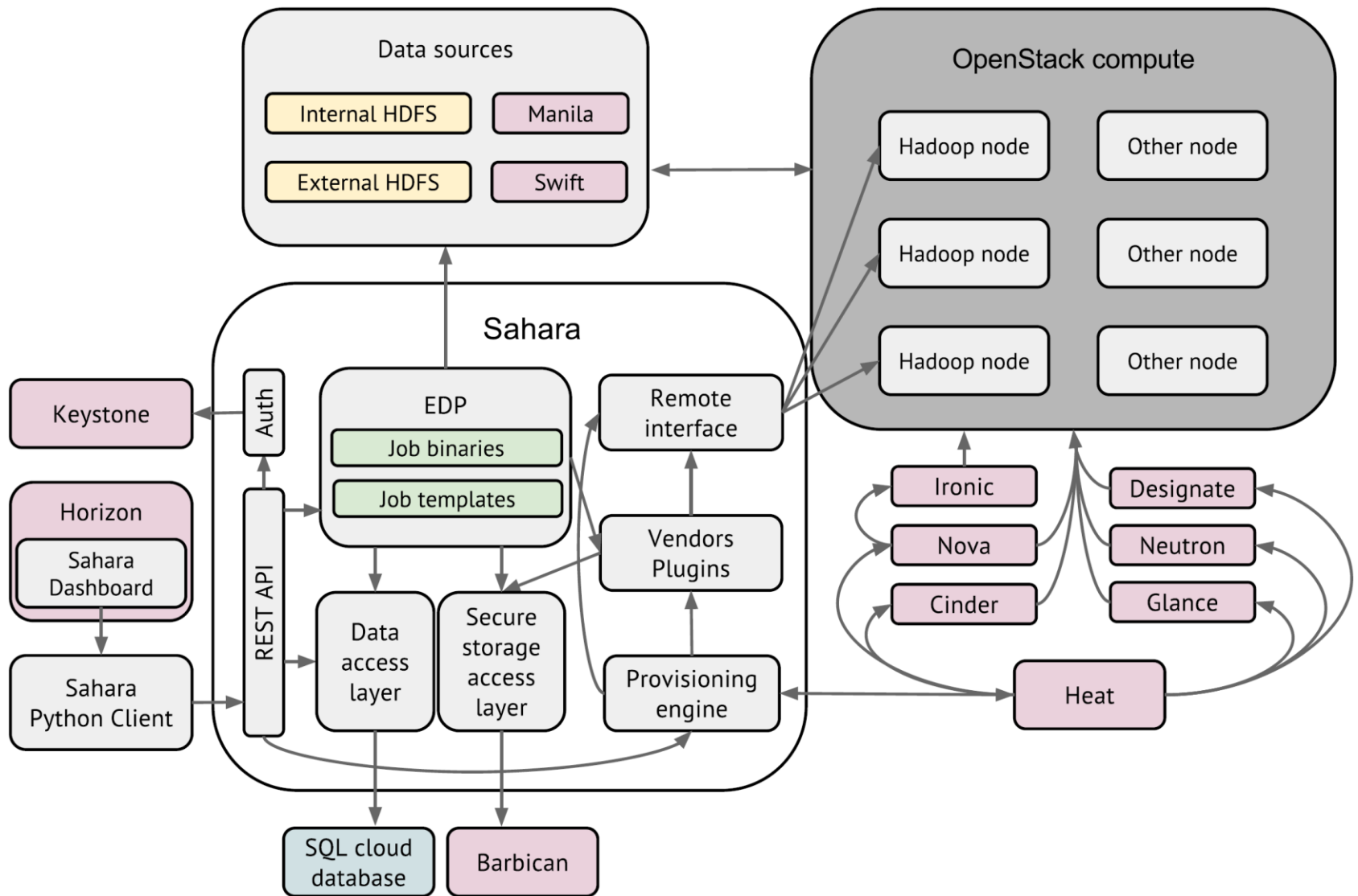
- **Cluster scaling functions**
- **Analytics as a Service (AaaS) functions**

Sahara is accessible by OpenStack in many ways
via: *dashboard, CLI or RESTful API*

SAHARA COMPONENTS



SAHARA COMPONENTS



MAPREDUCE, SPARK ANALYSIS

Our approach is architecture based

Semantics (Model),

External (black box),

Internal (open the box)

If you look at our approach, we got through steps:

- **Understand the principle and the architecture according to a user perspective** – the user must understand the whole functioning, and command internal user operations
- **The internal way of working and answering user requests**, but also how to deploy the internal support architecture toward the results
- **Understand all the internal details** that can grant the optimized requested behavior
- Widen the architectures to **other needed coordination tools**

SPARK ENGINE

It is not a modified version of Hadoop but a **separate, fast, MapReduce-like engine**:

New optimized version of Hadoop

- **In-memory data storage** for very fast iterative queries
- **General execution** of graphs and powerful optimizations
- Up to **40 times faster than Hadoop**

Compatible with Hadoop storage APIs

- Can read/write to any Hadoop-supported system, including HDFS, HBase, SequenceFiles, etc.

New way of working batch

SPARK PROJECT HISTORY

- Spark project started in 2009, open sourced 2010
- Spark started summer 2011, alpha April 2012
- In use at Berkeley, Princeton, Klout, Foursquare, Conviva, Quantifind, Yahoo! Research & others
- 200+ member meetup, 500+ watchers on GitHub

SPARK BASICS

Various types of data processing computations available in one single tool

- **Batch/streaming** analysis, **interactive** queries and **iterative** algorithms
- Previously, these would require **several different and independent tools**

Supports **several storage options and streaming inputs for parsing**

APIs available in Java, Scala, Python, R, ...

- Also R language supported, for data scientists with moderate programming experience

WHY SPARK?



Why a New Programming Model? An optimized batch

MapReduce greatly simplified **big data analysis**

But when it becomes popular, users wanted more:

- **More complex, multi-stage applications**
(e.g., iterative graph algorithms and machine learning)
- **More interactive ad-hoc queries**

Both multi-stage and interactive apps require faster **data sharing** across parallel jobs (**Resilient Distributed Datasets in memory RDDs**)

Use of **sharing** and **caching of data** with the goal **of speed**

SPARK AT A GLANCE

Leverages on in-memory data processing:

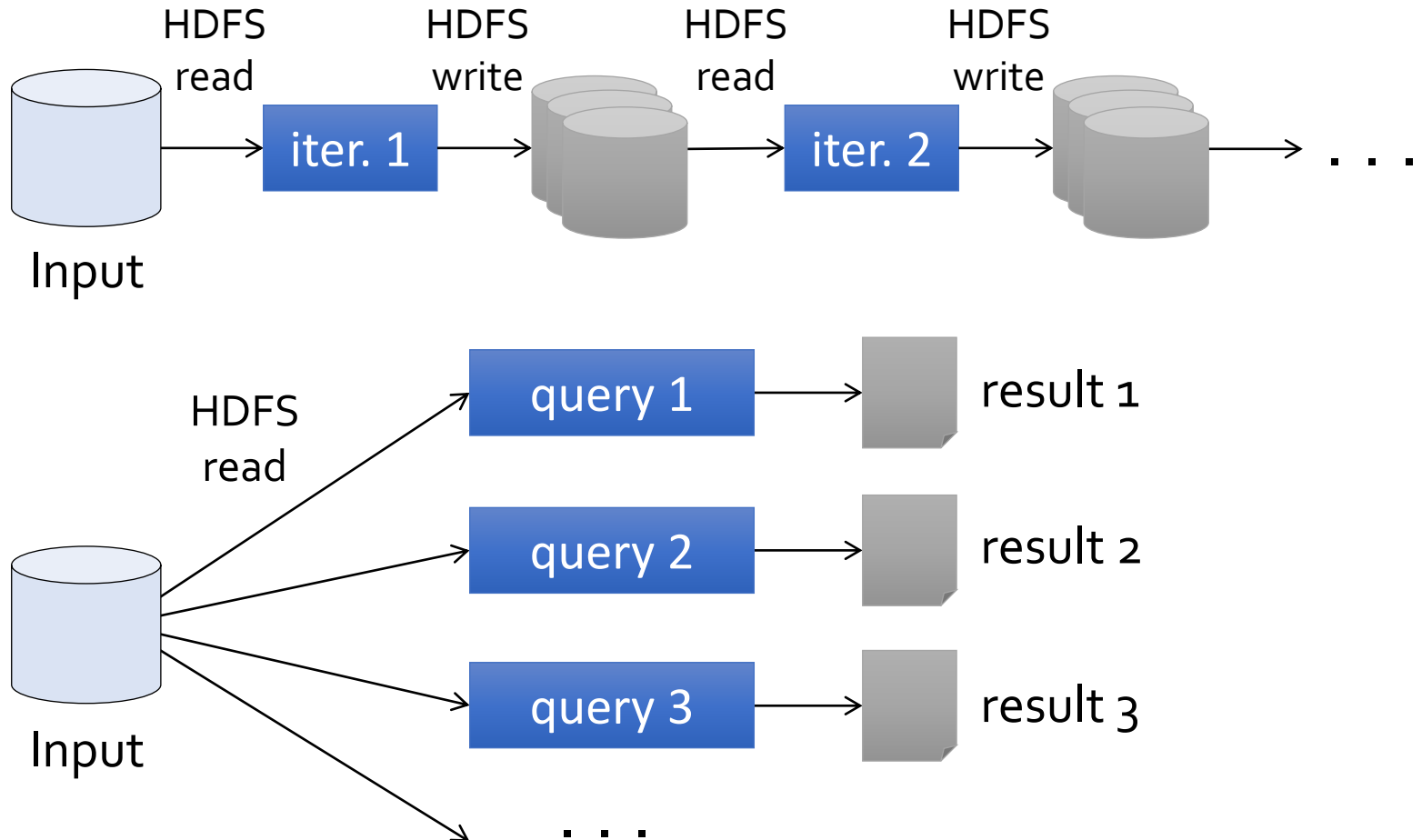
- Removes the MapReduce overhead of writing **intermediate results on disk**
- Fault-tolerance is still achieved through the concept of **lineage**

Master/Worker cluster architecture

- Easily deployable in most environments, including existing Hadoop clusters

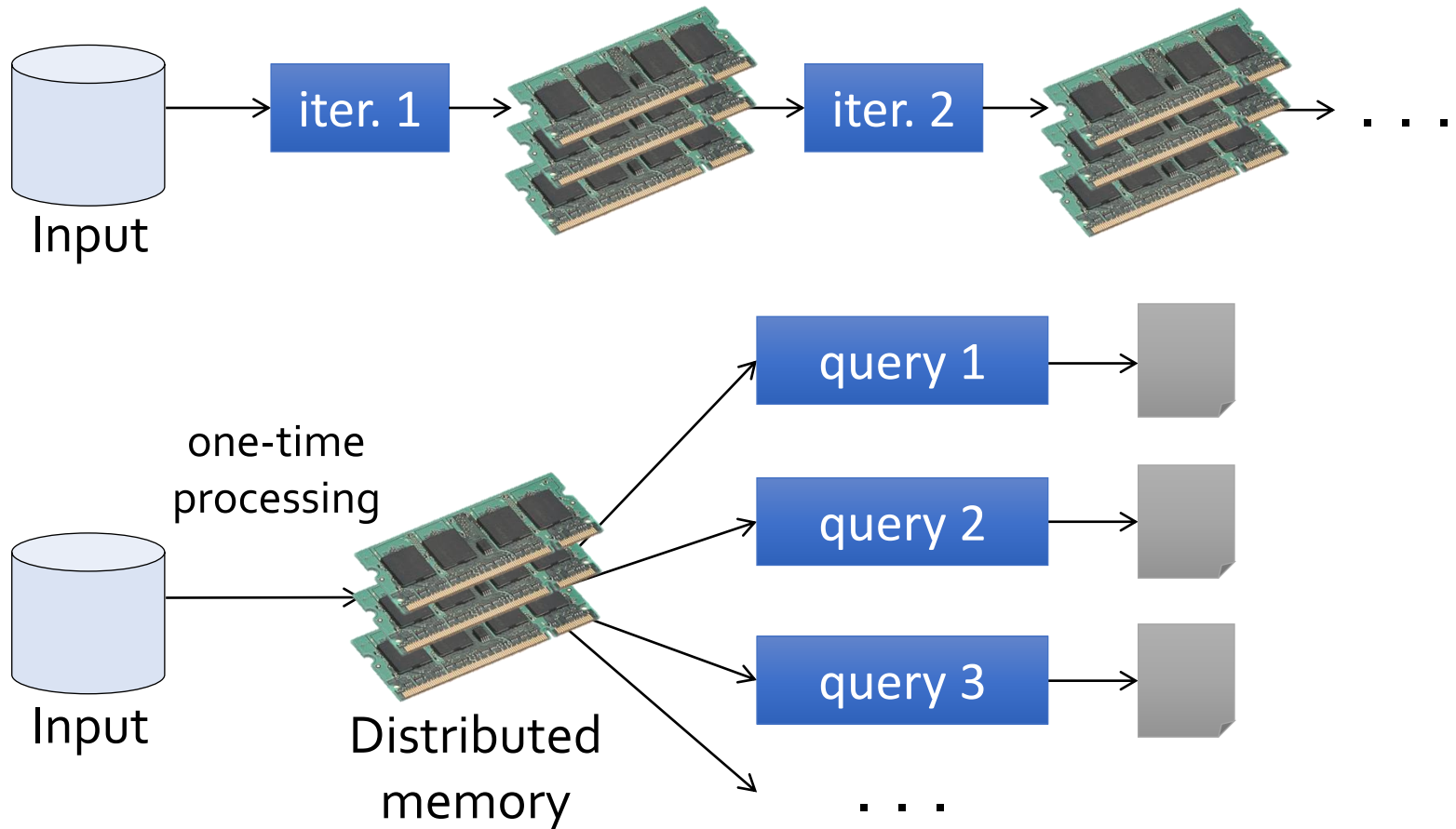
Widely configurable **for performance optimization**, both in terms of resource usage and application behavior

DATA SHARING IN MAPREDUCE



Slow due to replication, **serialization**, and **disk IO**

DATA SHARING IN SPARK



10-100× faster than network and disk

SPARK PROGRAMMING MODEL

Programs can be run both

- From **compiled sources**, with proper Spark dependencies, with the *Spark-submit* script
- *Interactively* from **Spark Shell**, a console available for Scala and Python languages

Key idea on data

Resilient Distributed Datasets (RDDs)

RDDs are

- **maintained in memory**
- **distributed, immutable collections of objects**
- **can be cached in memory across cluster nodes**

RDD PROGRAMMING MODEL

Two kinds of operations performed on ***Resilient Distributed Datasets***

Transformations that act on existing RDDs, by creating new ones

- Similar to Hadoop *map* tasks
- ***Lazily evaluated***

Actions that return results from input RDDs

- Similar to Hadoop *reduce* tasks
- ***Force immediate evaluation of pending transformations in the input RDD***

RDD TRANSFORMATION

In addition to **being lazily evaluated**, all **transformations** are computed again **on every action** requested

```
val lines = sc.textFile("data.txt")  
val lineLengths = lines.map(s => s.length)  
val totalLength = lineLengths.reduce((a, b) => a + b)
```



The diagram illustrates the lazy evaluation of RDD transformations. A speech bubble labeled "Transformation" points to the `map` operation in the second line of code. Another speech bubble labeled "Action" points to the `reduce` operation in the third line of code. This indicates that the `map` operation is only executed when the `reduce` operation is triggered.

Until the third line, no operation is performed

The `reduce()` will then force a read from the text file and the `map()` transformation

PERSISTING RDDs

In addition to *lazily evaluation*, all transformations are computed again *on every action* requested

Action

```
val lines = sc.textFile("data.txt")  
val lineLengths = lines.map(s => s.length)  
println(lineLengths.count())  
val totalLength = lineLengths.reduce((a, b) => a + b)
```

Transformation

Action

That effect is expensive, but can be avoided by using the *persist()* method

```
val lines = sc.textFile("data.txt")  
val lineLengths = lines.map(s => s.length)  
lineLengths.persist()
```

The **RDD data** read and mapped are saved for future actions

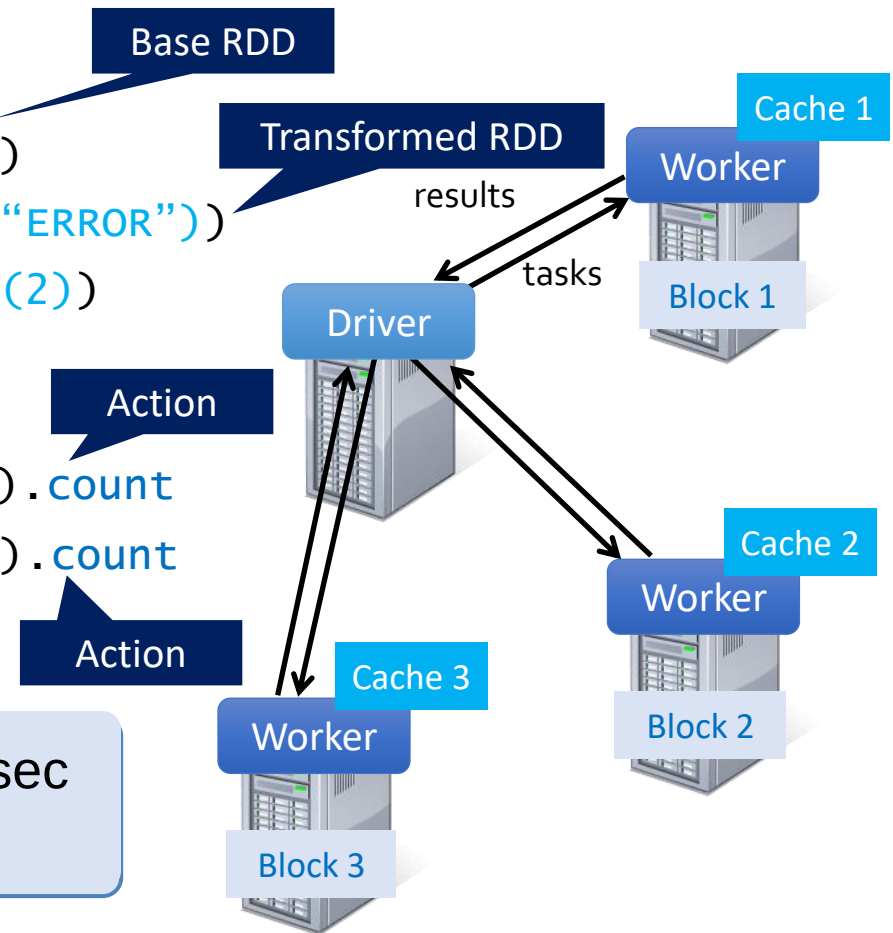
EXAMPLE LOG MINING

Load error messages from a log into memory, then interactively search for various patterns

```
lines = spark.textFile("hdfs://...")
errors = lines.filter(_.startsWith("ERROR"))
messages = errors.map(_.split('\t')(2))
cachedMsgs = messages.cache()

cachedMsgs.filter(_.contains("foo")).count
cachedMsgs.filter(_.contains("bar")).count
. . .
```

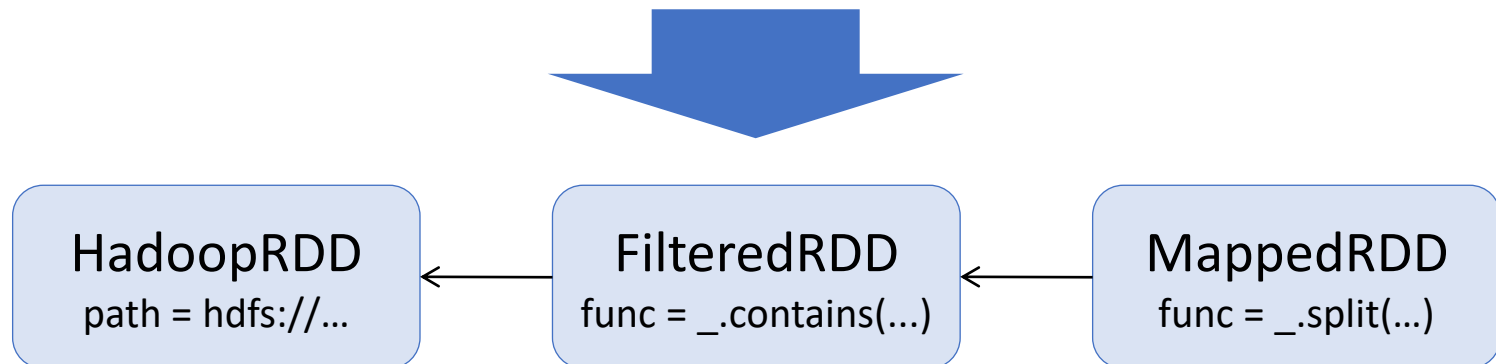
Result: scaled to 1 TB data in 5-7 sec
(vs 170 sec for on-disk data)



FAULT TOLERANCE

RDDs track the series of transformations used to build them (their *lineage*) to re-compute lost data

```
messages = textFile(...).filter(_.contains("error"))  
                        .map(_.split('\t')(2))
```



EXAMPLE: LOGISTIC REGRESSION

Load data in memory once

```
val data =  
  spark.textFile(...).map(readPoint).cache()
```

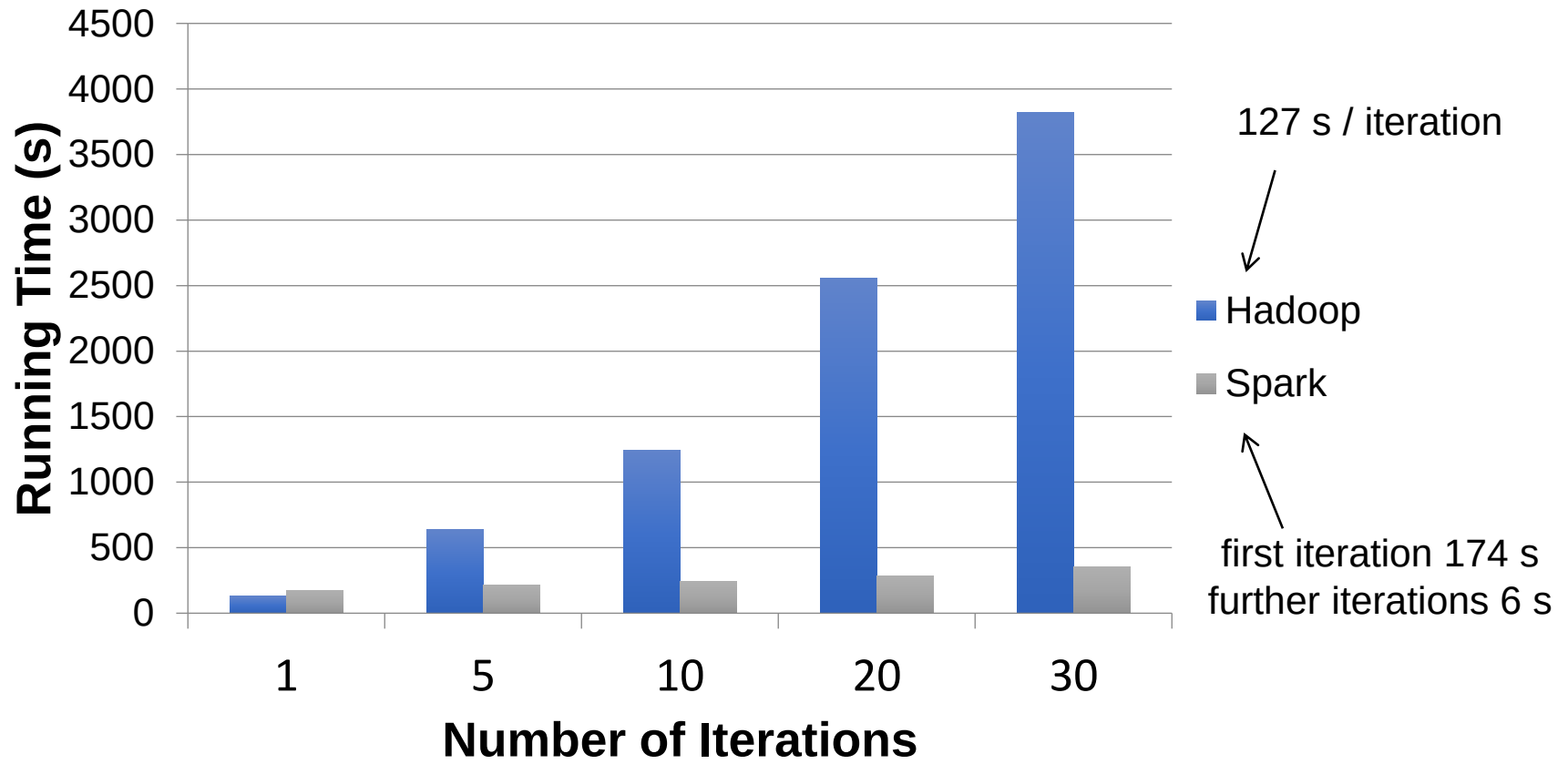
```
var w = Vector.random(D)
```

Initial parameter vector

```
for (i <- 1 to ITERATIONS) {  
  val gradient = data.map( p =>  
    (1 / (1 + exp(-p.y*(w dot p.x))) - 1)  
    * p.y * p.x  
  ).reduce(_ + _)  
  w -= gradient  
}  
println("Final w: " + w)
```

Repeated MapReduce steps
to do gradient descent

LOGISTIC REGRESSION PERFORMANCES



SUPPORTED OPERATORS

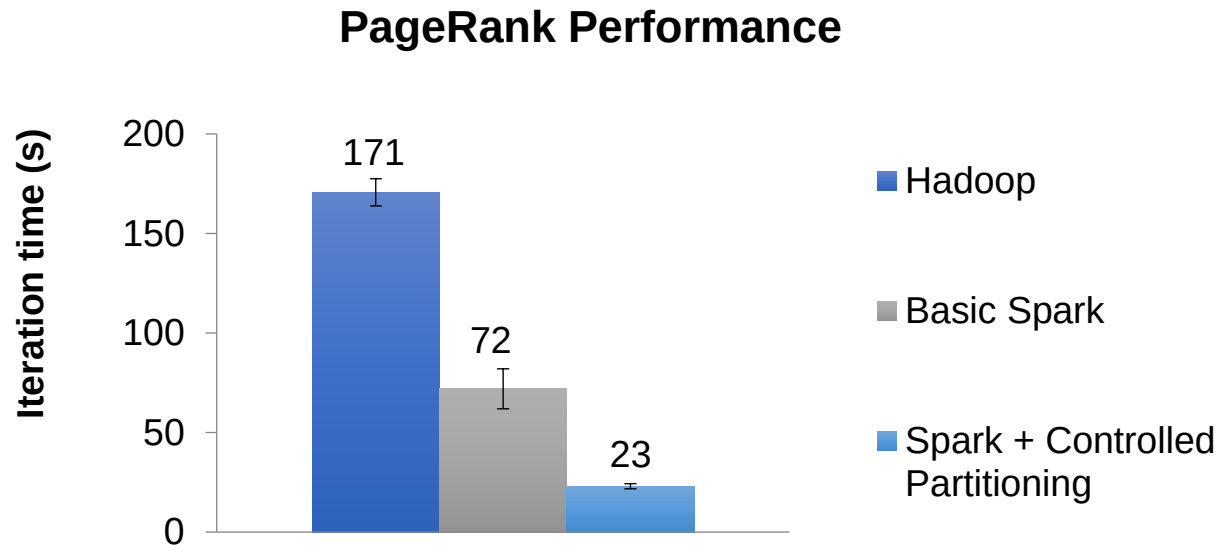
- `map`
- `filter`
- `groupBy`
- `sort`
- `join`
- `leftOuterJoin`
- `rightOuterJoin`
- `reduce`
- `count`
- `reduceByKey`
- `groupByKey`
- `first`
- `union`
- `cross`
- `sample`
- `cogroup`
- `take`
- `partitionBy`
- `pipe`
- `save`
- `...`

OTHER ENGINE FEATURES

General **graphs of operators** (e.g. map-reduce-reduce)

Hash-based reduces (faster than Hadoop sort)

Controlled data partitioning adapted to lower communication



SPARK ARCHITECTURE

Once submitted, Spark programs create ***directed acyclic graphs (DAGs)*** of all transformations and actions, internally optimized for the execution

The graph is then split into **stages**, in turn composed by **tasks**, the smallest unit of work

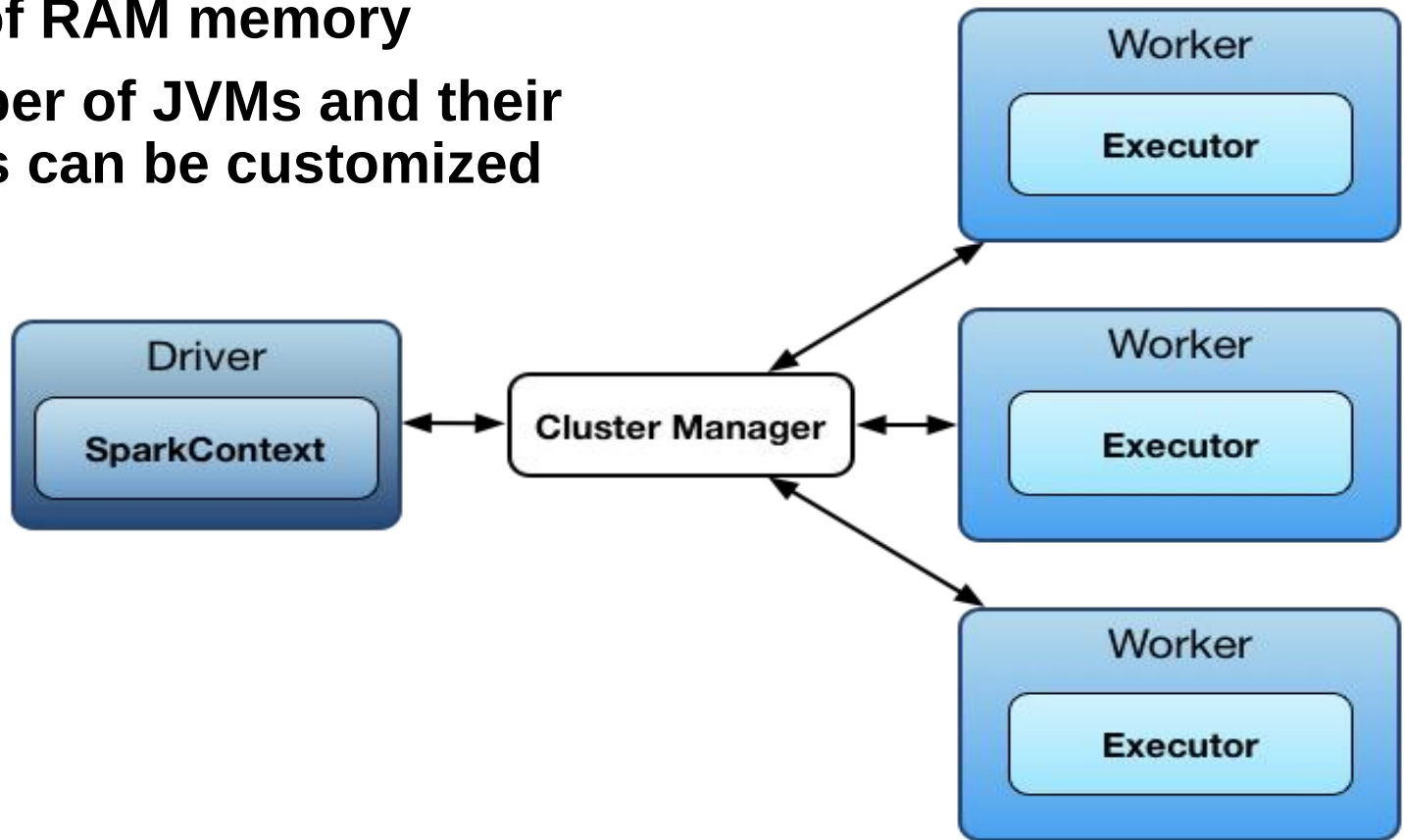
The Spark support is a **master/slave** system composed by:

- **Driver**, central coordinator node running the *main()* method of the program and dispatching tasks
- **Cluster Master**, node that launches and manages actual executors
- **Executors**, responsible for running tasks

SPARK ARCHITECTURE

Each executor spawns at least one dedicated **JVM**, to which a certain share of resources is assigned, in terms of:

- **Number of CPU threads**
- **Amount of RAM memory**
- **The number of JVMs and their resources can be customized**



SPARK DEPLOYMENT

Spark can be **deployed in a standalone cluster**, i.e., its own cluster master independently launches and manages its executors

However, Spark can rely upon **external resource managers**, such as:

- **Hadoop YARN (already seen before...)**
- **Apache MESOS (fine-grained sharing, ...)**

These others can provide richer functionalities, such as resource scheduling queues, not available in the standalone mode

SPARK PROVISIONING

Spark can produce a **very large result**

How can you easily manage it?

In general you can come out with counting words,

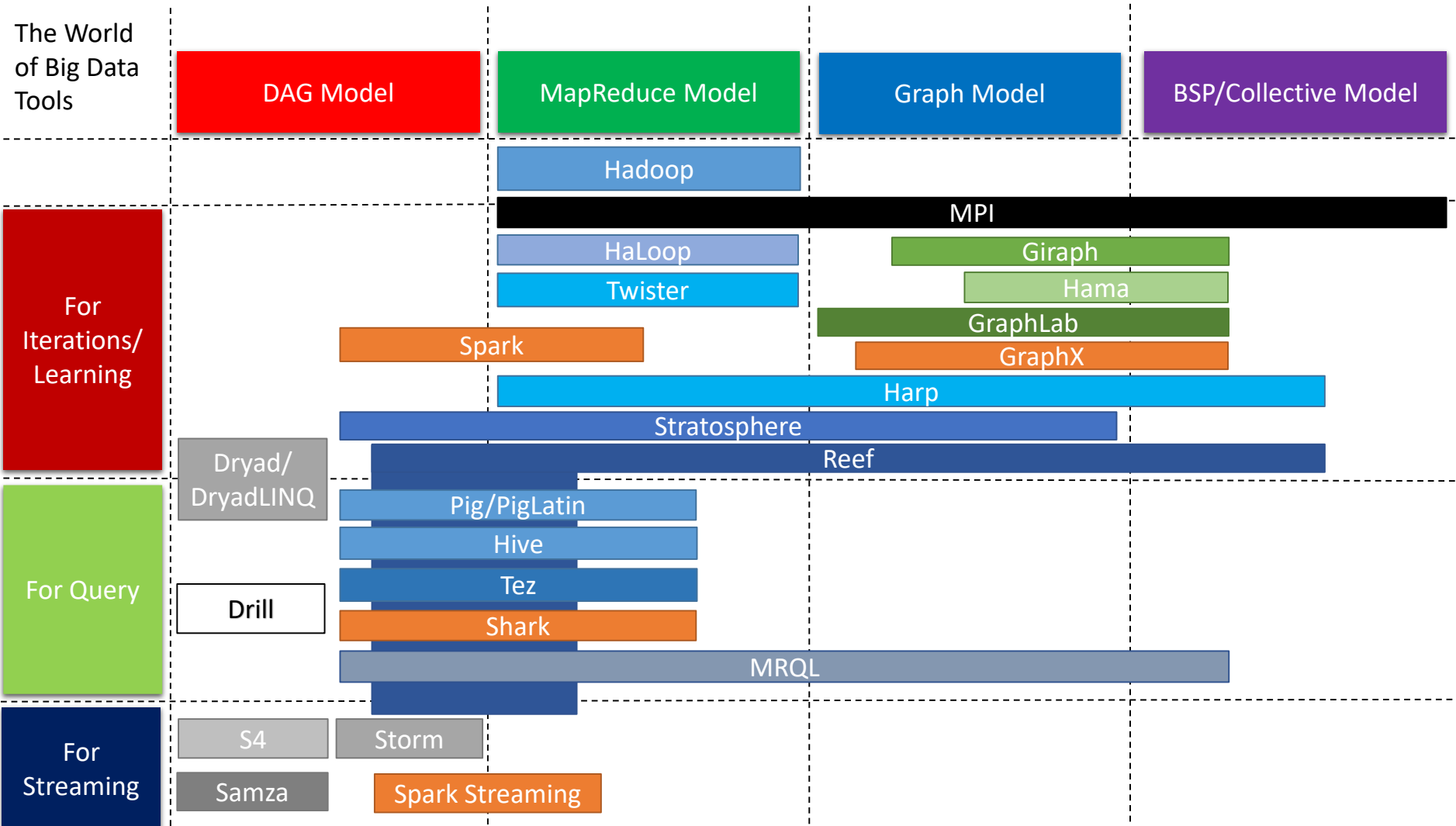
but it can also have **links to larger aggregations** (Document at url?)

not easy to be managed, better automated

So the experience asks for a new way of accessing some large
produced batch data

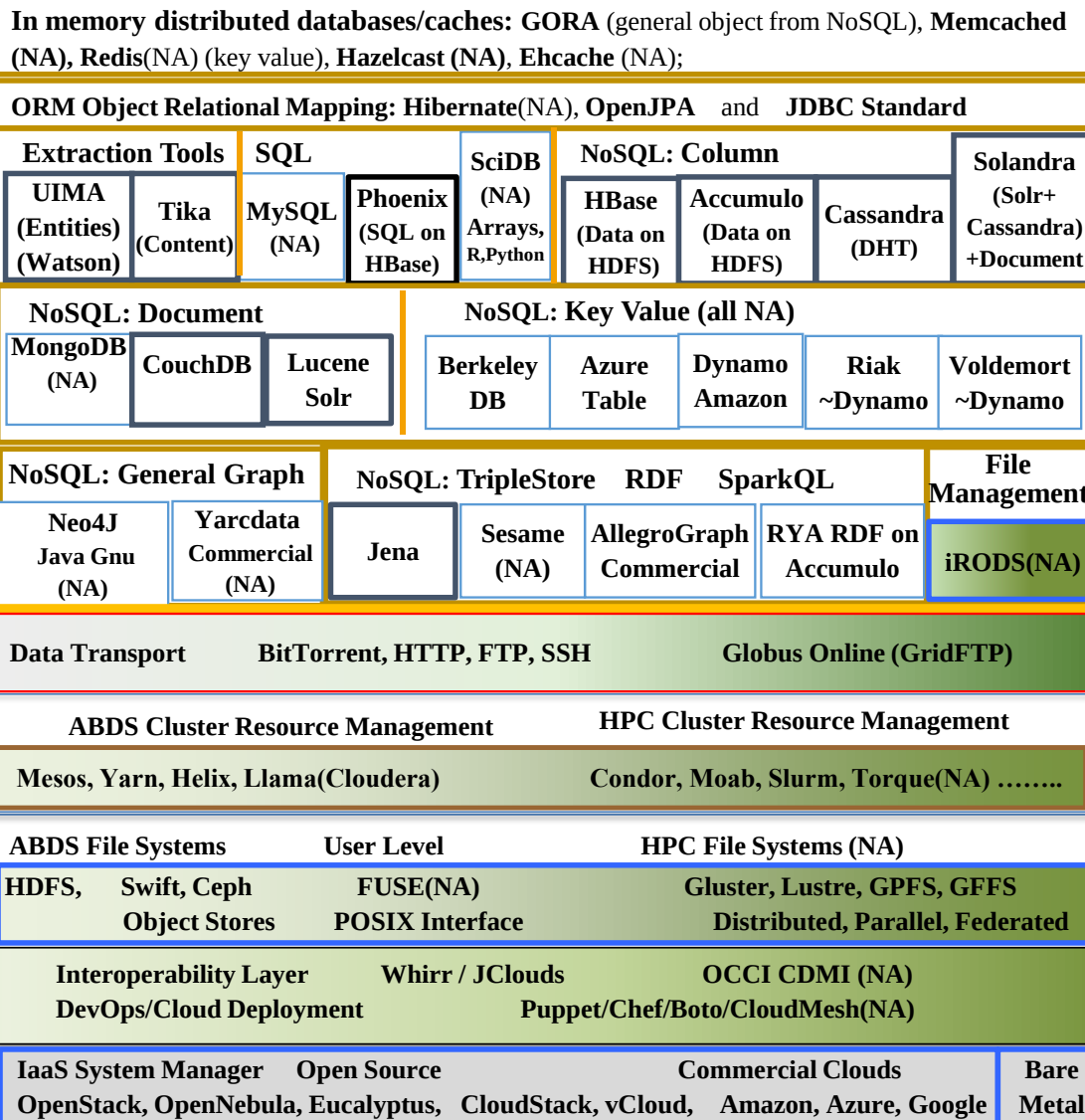
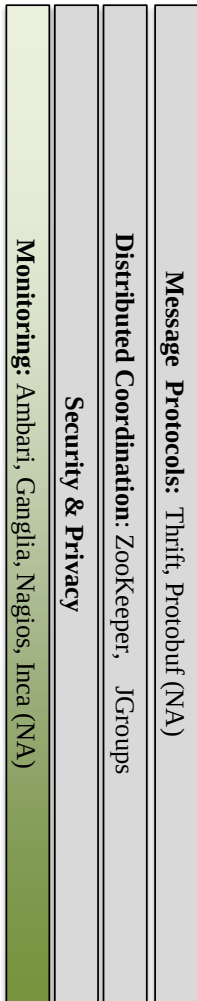
Going into NoSQL repositories

THE BIG DATA TOOLS ECOSYSTEM



A LAYERED ARCHITECTURE VIEW

Cross Cutting Capabilities



- NA – Non-Apache projects
- Green layers are Apache/ Commercial Cloud (light) to HPC (darker) integration layers

IT REMOTE INFRASTRUCTURE

Access via Web, many protocols

Metamodel → remote access to Cloud, DC, System

Black box approach

REMOTE DASHBOARD

User visible model of solution

User model (Map and Reduce functions, ...)

Internal support

Architecture decisions and resource support

ANALYSIS OF BIG DATA AND SERVICES

RESOURCES as UNIFYING concepts

ISSUES for RESOURCES

- Resource Sharing (multicast)
- Resource Distribution (events)
- Resource Synchronization
- Resource Replication
- Resource Control

- Resource Configuration
- Resource Timing

ANALYSIS OF BIG DATA AND SERVICES

RESOURCES as UNIFYING concepts

ISSUES for RESOURCES

- Resource Sharing (multicast)
- Resource Distribution (events)
- Resource Synchronization
- Resource Replication
- Resource Control

- Resource Configuration
- Resource Timing

RUN TIME

STATIC

BEFORE RUN TIME

IT PROPERTIES

Technical properties

- **Dynamicity and adaptability**
- **Fault tolerance or Replication (availability and reliability)**
- **Loose Consistency**
- **Group communication**
- **Data configuration and access**
- **Resource life cycle support**

- **Transparency**
- **Low intrusion**
- **Time awareness**
- **Simplicity**